okonomi — Visão

Artefato 2 do Trabalho Prático de Engenharia de Software (2026.2): documento de visão e escopo. Segue a estrutura do modelo `vision_tpl.odt` (`docs/RF/Templates/`). Rastreado pela issue #26; atende ao Critério de Avaliação 02.

Versão	Data	Autor	Descrição
1.0	28/09/2026	José Wilson Barbosa de Sousa Júnior	Versão inicial.
1.1	04/11/2026	José Wilson Barbosa de Sousa Júnior	Plataforma e metas de compatibilidade e desempenho alinhadas ao artefato 3 (RNF-10, RNF-17, RNF-36); comparação com as ferramentas concorrentes na seção 2.2.

1. Introdução

O **okonomi** é um sistema web de **gestão de requisitos para projetos que usam histórias de usuário**. Ele reúne em um só lugar o product backlog, os sprint backlogs, as histórias, os épicos, os critérios de aceitação e a priorização de cada história, e impõe pela própria estrutura dos dados os formatos e escalas que as práticas ágeis recomendam.

Este documento define o problema, o público, as partes interessadas, o ambiente de uso e as funcionalidades do sistema em alto nível. O detalhamento fica nos demais artefatos: histórias de usuário (artefato 4), requisitos não funcionais (artefato 3), arquitetura (artefato 5) e banco de dados (artefato 7).

2. Posicionamento

2.1 Declaração do problema

O problema de	registrar requisitos como histórias de usuário em ferramentas genéricas — planilhas, quadros de tarefas, documentos de texto — que não conhecem o formato de uma história nem as escalas de estimativa e priorização
afeta	Product Owners, Scrum Masters e times de desenvolvimento de equipes ágeis pequenas, além de estudantes e professores de Engenharia de Software
cujo impacto é	histórias escritas em texto livre, sem papel, ação ou benefício claros; critérios de aceitação ausentes ou vagos; story points fora da escala combinada; prioridades decididas sem critério explícito; e a mesma história copiada entre backlogs até ninguém saber qual versão vale
uma solução bem-sucedida	garantiria que toda história siga o modelo *Como um … eu quero … para …* e todo critério o modelo *Dado … quando … então …*; limitaria story points à escala de Fibonacci; calcularia a prioridade RICE automaticamente; e manteria cada história em exatamente um backlog, movendo-a entre product e sprint backlogs sem duplicação

2.2 Declaração de posição do produto

Para	equipes ágeis pequenas e turmas de Engenharia de Software

que	precisam escrever, estimar e priorizar histórias de usuário de forma disciplinada
o okonomi	é um sistema web de gestão de requisitos
que	torna os formatos de história e de critério de aceitação, a escala de story points e os domínios de MoSCoW e RICE impossíveis de violar, porque estão na estrutura dos dados e não só na interface
diferente de	ferramentas de gestão de projetos de propósito geral, como Jira, Trello e planilhas, que tratam uma história como um cartão de texto livre e deixam formato, escala e priorização a cargo da disciplina de cada pessoa
nosso produto	é focado apenas em requisitos ágeis, é simples de aprender e calcula a pontuação RICE de cada história, ordenando o backlog por um critério explícito

Comparação com as alternativas. As ferramentas mais usadas por equipes ágeis resolvem um problema mais amplo, a gestão do projeto inteiro, e por isso tratam a história de usuário como mais um tipo de item:

Ferramenta	Como trata histórias e priorização	Custo e complexidade
Jira (Atlassian)	História é um tipo de *issue* com texto livre; story points são um número qualquer; MoSCoW e RICE só com campos personalizados ou extensões pagas.	Gratuito até 10 usuários, pago acima disso; muitas configurações, curva de aprendizado alta.
Trello (Atlassian)	Cartões genéricos em colunas; não há conceito de história, critério de aceitação, backlog ou sprint sem *Power-Ups*.	Plano gratuito limitado; simples, mas sem nenhuma regra de requisitos.
Taiga	Tem histórias, épicos, sprints e pontos, mais próximo do okonomi; critérios de aceitação são texto livre e não há MoSCoW nem RICE.	Código aberto; hospedagem própria ou plano pago; voltado a vários times e papéis.
Azure Boards (Microsoft)	*Work items* configuráveis por processo (Agile, Scrum, CMMI); prioridade é um número de 1 a 4; RICE só com campos e consultas personalizadas.	Gratuito até 5 usuários; integrado ao ecossistema Azure DevOps, pesado para quem só quer o backlog.

O okonomi ocupa o espaço que essas ferramentas deixam aberto:

- **gratuito e de código aberto** (licença MIT), sem limite de usuários;
- **focado só em histórias de usuário**: formato *Como um … eu quero … para …* e critérios *Dado … quando … então …* garantidos pela estrutura dos dados, e não por disciplina;
- **MoSCoW e RICE nativos**, com domínios fechados e cálculo automático da pontuação, sem extensões nem campos personalizados;
- **simples, para um único time**: cada pessoa mantém os próprios projetos, sem configuração de fluxos, permissões ou esquemas de campos.

O okonomi não pretende competir com as ferramentas completas de gestão de projetos. Seu espaço é o de uma ferramenta **especializada e didática**: pequena o bastante para ser adotada em uma tarde, rigorosa o bastante para ensinar a forma correta de escrever e priorizar requisitos.

3. Partes interessadas

Nome	Descrição	Responsabilidades
Professor da disciplina	Cliente do trabalho prático; autor do enunciado.	Define os requisitos (seção 2 do enunciado) e os critérios de avaliação; avalia os artefatos e o protótipo.

Nome	Descrição	Responsabilidades
Equipe de desenvolvimento	José Wilson Barbosa de Sousa Júnior (matrícula 241024259).	Elicita e especifica requisitos; projeta, implementa, testa e implanta o sistema; mantém o quadro Kanban; produz os artefatos.
Product Owner (usuário)	Responsável pelo produto em uma equipe ágil.	Cria projetos e o product backlog; escreve histórias e critérios de aceitação; prioriza com MoSCoW e RICE; organiza histórias em épicos.
Scrum Master (usuário)	Facilitador do processo da equipe.	Cria sprint backlogs; acompanha a movimentação das histórias entre backlogs; zela pelo formato das histórias.
Time de desenvolvimento (usuário)	Quem implementa as histórias.	Estima story points; consulta histórias e critérios de aceitação da sprint; ajuda a definir o esforço usado no RICE.

4. Ambiente do usuário

- **Tamanho das equipes.** Equipes ágeis pequenas, de três a nove pessoas, ou estudantes trabalhando sozinhos ou em grupos de até cinco. Cada pessoa tem sua própria conta e vê apenas os seus projetos.
- **Ciclo de trabalho.** O trabalho se organiza em sprints de uma a quatro semanas. No planejamento de cada sprint, histórias são movidas do product backlog para o sprint backlog; durante a sprint, histórias e critérios são consultados e refinados. Uma sessão típica dura de alguns minutos (ajustar uma estimativa) a uma hora (refinamento do backlog).
- **Plataforma.** Navegador web atual (Chrome, Edge, Firefox ou Safari), no computador ou no celular, com acesso à internet. O uso principal é no computador de mesa ou notebook, mas todas as telas funcionam a partir de 375 px de largura, sem rolagem horizontal (RNF-10 e RNF-36 do artefato 3). Não há instalação no dispositivo do usuário nem aplicativo móvel nativo: no celular, o acesso é pelo próprio navegador.
- **Restrições do ambiente.** Uso em escritório ou em casa; nenhuma restrição especial de mobilidade, conectividade intermitente ou ambiente externo.
- **Outras aplicações.** As equipes costumam usar também um repositório de código (como o GitHub) e um canal de comunicação. O okonomi **não** precisa se integrar a elas nesta versão.

5. Visão geral do produto

5.1 Necessidades e funcionalidades

Prioridade na escala MoSCoW (**M** = *must have*). Todas as funcionalidades abaixo são requisitos do enunciado e, por isso, obrigatórias. A entrega planejada é o marco **M4 — Protótipo**.

Necessidade	Prioridade	Funcionalidades	Requisito
Usar o sistema pelo navegador, sem instalação.	M	Interface gráfica web.	RF01
Manter os próprios dados protegidos.	M	Criação de conta; autenticação; acesso aos serviços somente após autenticar.	RF02, RF03
Separar o trabalho de produtos diferentes.	M	Criar, consultar, alterar e excluir projetos.	RF04

Necessidade	Prioridade	Funcionalidades	Requisito
Ter uma fonte única de tudo que o produto precisa.	M	Um único product backlog por projeto, com criação, consulta, alteração e exclusão.	RF05
Planejar o trabalho de cada iteração.	M	Vários sprint backlogs por projeto, com criação, consulta, alteração e exclusão.	RF06
Registrar requisitos do ponto de vista do usuário.	M	Criar, consultar, alterar e excluir histórias; toda história nasce no product backlog; formato *Como um … eu quero … para …*.	RF07, RF09
Levar histórias para uma sprint e devolvê-las.	M	Mover histórias entre backlogs do mesmo projeto.	RF08
Agrupar histórias por objetivo maior.	M	Criar, consultar, alterar e excluir épicos; vincular histórias a épicos do mesmo projeto.	RF10, RF11
Saber quando uma história está pronta.	M	Criar, consultar, alterar e excluir critérios de aceitação; formato *Dado … quando … então …*.	RF12, RF13
Estimar o esforço de cada história.	M	Atribuir story points na escala 0, 1, 2, 3, 5, 8, 13, 21, 34, 55.	RF14, RF15
Decidir o que fazer primeiro.	M	Etiqueta MoSCoW (M, S, C, W); critério RICE com domínios fechados; cálculo da pontuação (R × I × C) / E.	RF16–RF19

6. Outros requisitos do produto

Resumo dos requisitos não funcionais. A especificação completa, com normas, métricas e licenças, é o artefato 3 (#27).

Requisito	Prioridade	Entrega planejada
Usabilidade — um usuário novo cria um projeto, uma história e um critério sem consultar manual; mensagens de erro em português, junto ao campo com problema.	M	M4
Segurança — senhas guardadas como *hash* (bcrypt); cookie de sessão `secure`, `httponly` e `SameSite=Lax`; HTTPS obrigatório em produção; cada usuário acessa apenas os próprios dados; mensagem de falha de login não revela se o e-mail existe.	M	M4
Integridade dos dados — os formatos e domínios dos requisitos funcionais são garantidos pelo banco de dados, não só pela interface.	M	M0 (concluído em #24)
Compatibilidade — funciona nas versões atuais de Chrome, Edge, Firefox e Safari, no computador ou no celular, em telas a partir de 375 px de largura, sem rolagem horizontal (RNF-10, RNF-36).	M	M4
Desempenho — o servidor responde às páginas em até 500 ms no percentil 95, com projetos de até 200 histórias (RNF-17).	S	M4
Conformidade — dados pessoais (e-mail) tratados conforme a LGPD; acessibilidade orientada pela WCAG 2.2.	S	M4
Licenciamento — apenas dependências de código aberto com licenças permissivas (MIT, BSD, PostgreSQL License).	M	M5
Documentação — README com instruções de instalação e execução; artefatos 1 a 9 entregues em PDF.	M	M5

Requisito	Prioridade	Entrega planejada
Qualidade do código — integração contínua com análise estática (RuboCop, Brakeman), auditoria de dependências e testes automatizados em cada pull request.	M	M0 (concluído)

Suposições e dependências. O sistema supõe um servidor Linux com PostgreSQL 17 e acesso à internet para implantação; se essa plataforma mudar, as seções 6 e 7 deste documento precisam ser revistas.

7. Elementos da solução

A equipe propõe uma **aplicação web monolítica** — apresentação, regras de negócio e acesso a dados no mesmo sistema — por ser a forma mais simples de entregar um protótipo integrado e fácil de implantar.

Elemento	Escolha	Por quê
Linguagem e framework	**Ruby 3.4** e **Ruby on Rails 8.1**	Traz autenticação, filas, cache e implantação prontos, sem dependências extras.
Interface	HTML renderizado no servidor com **Hotwire** (Turbo e Stimulus) e **Tailwind CSS**	Interface responsiva sem uma aplicação JavaScript separada.
Banco de dados	**PostgreSQL 17**	Suporta índice único parcial (um product backlog por projeto), chaves estrangeiras compostas (histórias só se movem e só se vinculam a épicos dentro do mesmo projeto), restrições `CHECK` (escalas de story points, MoSCoW e RICE) e `decimal` exato (Impact 0,25).
Filas, cache e websockets	**Solid Queue**, **Solid Cache**, **Solid Cable**	Funcionam sobre o próprio PostgreSQL, sem Redis.
Implantação	**Kamal** com contêineres Docker, servidor **Puma** atrás do **Thruster**	Um único servidor Linux basta para pôr o sistema em produção.
Qualidade	**Minitest** e **Capybara**; **GitHub Actions**	Cada critério de aceitação vira um teste; nenhuma mudança entra sem CI verde.

As regras centrais do domínio — formato da história em três campos (papel, ação, benefício), formato do critério em três campos (contexto, ação, resultado), escalas fechadas e cálculo do RICE — já estão implementadas no modelo de dados (#24) e detalhadas nos artefatos 5 e 7.

8. Rastreabilidade com o Critério de Avaliação 02

Item do critério	Seção
Descrito o problema resolvido pelo sistema de software.	2.1
Descrita a posição que o sistema pretende ocupar no mercado.	2.2
Descritas as partes interessadas e suas responsabilidades.	3
Descrito o ambiente de trabalho dos futuros usuários.	4
Descritas as necessidades atendidas pelo sistema.	5.1

Item do critério	Seção
Descritas resumidamente as funcionalidades a serem providas.	5.1
Descritos resumidamente requisitos não funcionais.	6
Descritos resumidamente elementos da solução proposta.	7